Building PES-VCS: A Version Control System from Scratch

Objective: Build a local version control system that tracks file changes, stores snapshots efficiently, and supports commit history. Every component maps directly to operating system and filesystem concepts.

Platform: Ubuntu 24.04

Name | **SRN** : Abhigyan Dutta | PES2UG24CS019

Getting Started

Prerequisites

```
sudo apt update && sudo apt install -y gcc build-essential libssl-dev
```

Building

```
make          # Build the pes binary
make all      # Build pes + test binaries
make clean    # Remove all build artifacts
```

Author Configuration

PES-VCS reads the author name from the `PES_AUTHOR` environment variable:

```
export PES_AUTHOR="Your Name <PESXUG24CS042>"
```

If unset, it defaults to `"PES User <pes@localhost>"`.

File Inventory

File	Role	Your Task (Done)
`pes.h`	Core data structures and constants	Do not modify
`object.c`	Content-addressable object store	Implemented `object_write`, `object_read`
`tree.h`	Tree object interface	Do not modify
`tree.c`	Tree serialization and construction	Implemented `tree_from_index`
`index.h`	Staging area interface	Do not modify
`index.c`	Staging area (text-based index file)	Implemented `index_load`, `index_save`, `index_add`
`commit.h`	Commit object interface	Do not modify
`commit.c`	Commit creation and history	Implemented `commit_create`
`pes.c`	CLI entry point and command dispatch	Do not modify
`test_objects.c`	Phase 1 test program	Do not modify
`test_tree.c`	Phase 2 test program	Do not modify
`test_sequence.sh`	End-to-end integration test	Do not modify

`Makefile`	Build system	Do not modify

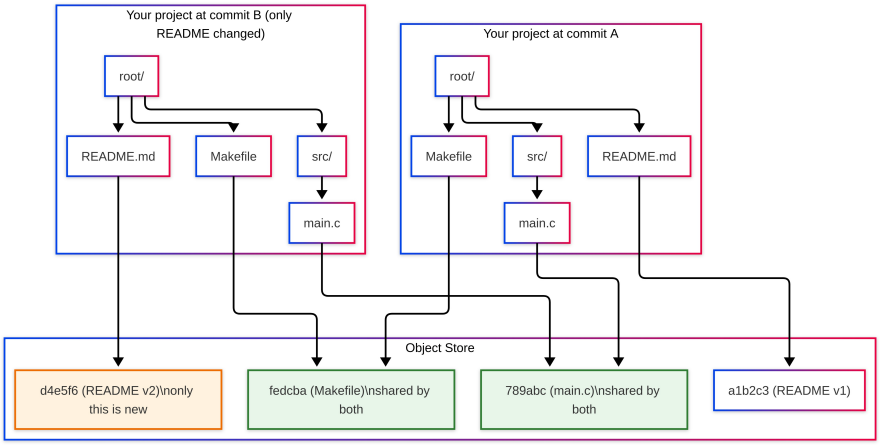
Understanding Git: What You're Building

Before writing code, understand how Git works under the hood. Git is a content-addressable filesystem with a few clever data structures on top. Everything in this lab is based on Git's real design.

The Big Picture

When you run `git commit`, Git doesn't store "changes" or "diffs." It stores **complete snapshots** of your entire project. Git uses two tricks to make this efficient:

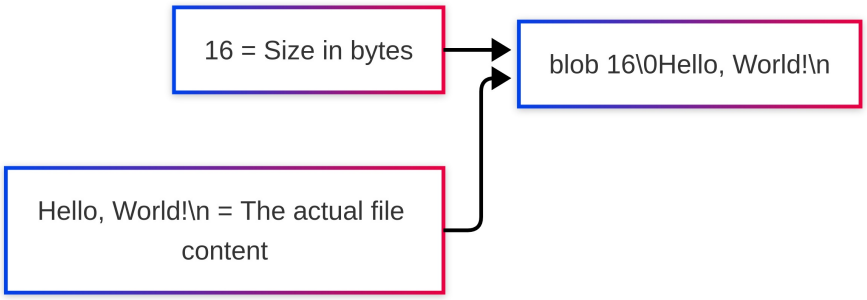
1. **Content-addressable storage:** Every file is stored by the SHA hash of its contents. Same content = same hash = stored only once.
2. **Tree structures:** Directories are stored as "tree" objects that point to file contents, so unchanged files are just pointers to existing data.



The Three Object Types

1. Blob (Binary Large Object)

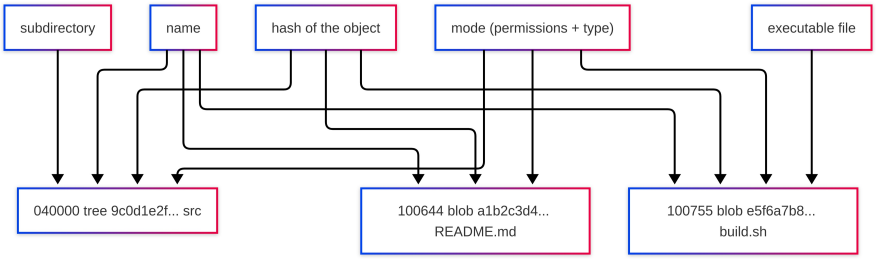
A blob is just file contents. No filename, no permissions - just the raw bytes.



The blob is stored at a path determined by its SHA-256 hash. If two files have identical contents, they share one blob.

2. Tree

A tree represents a directory. It's a list of entries, each pointing to a blob (file) or another tree (subdirectory).



Mode values:

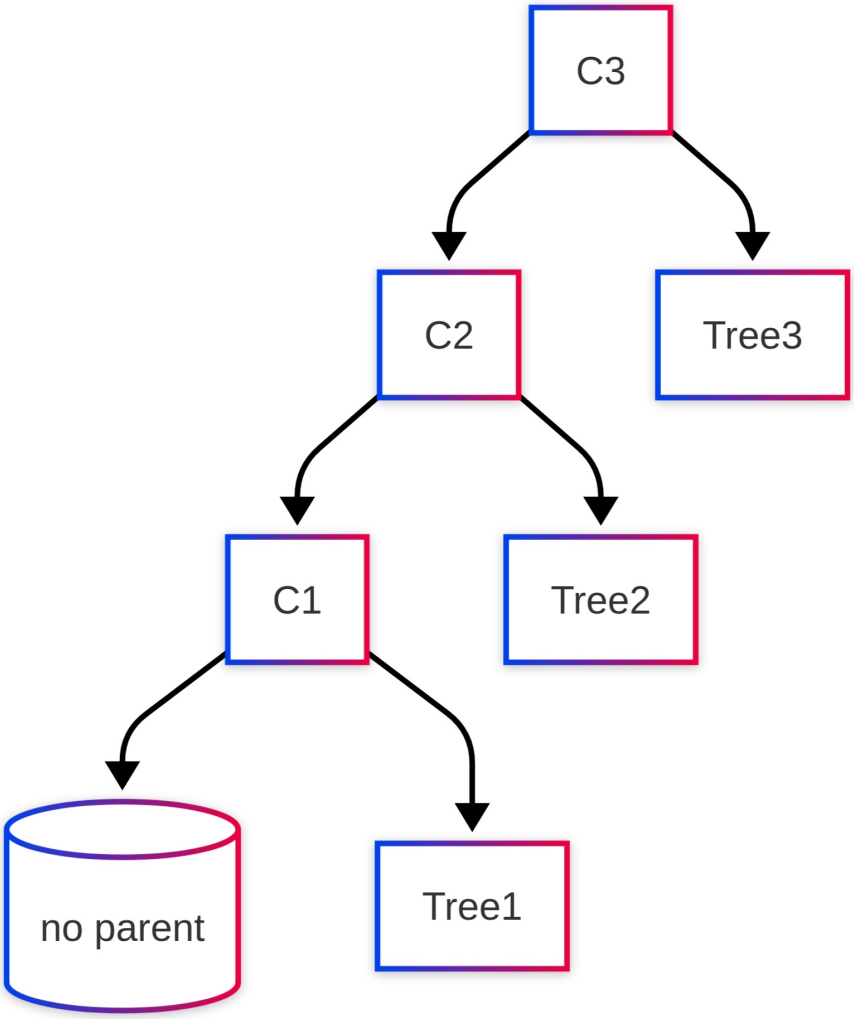
- `100644` - regular file, not executable
- `100755` - regular file, executable
- `040000` - directory (tree)

3. Commit

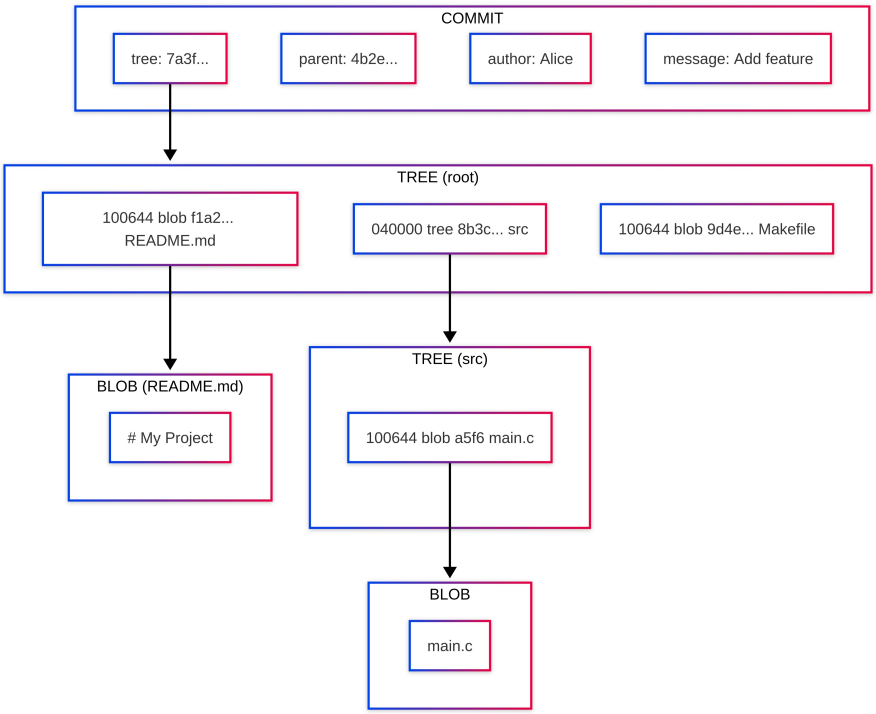
A commit ties everything together. It points to a tree (the project snapshot) and contains metadata.

```
tree 9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d
parent a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
author Alice <alice@example.com> 1699900000
committer Alice <alice@example.com> 1699900000

Add new feature
```

The parent pointer creates a linked list of history:



How Objects Connect



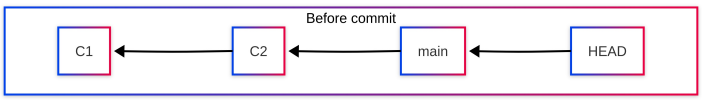
References and HEAD

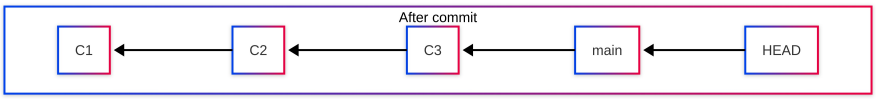
References are files that map human-readable names to commit hashes:

```
.pes/
├── HEAD                    # "ref: refs/heads/main"
└── refs/
    └── heads/
        └── main            # Contains: a1b2c3d4e5f6...
```

HEAD points to a branch name. The branch file contains the latest commit hash. When you commit:

1. Git creates the new commit object (pointing to parent)
2. Updates the branch file to contain the new commit's hash
3. HEAD still points to the branch, so it "follows" automatically





The Index (Staging Area)

The index is the "preparation area" for the next commit. It tracks which files are staged.

```
Working Directory          Index              Repository (HEAD)
─────────────────          ─────              ─────────────────
README.md (modified) ─── pes add ──► README.md (staged)
src/main.c                            src/main.c          ──► Last
commit's
Makefile                               Makefile
snapshot
```

The workflow:

1. `pes add file.txt` → computes blob hash, stores blob, updates index
2. `pes commit -m "msg"` → builds tree from index, creates commit, updates branch ref

Content-Addressable Storage

Objects are named by their content's hash:

```
# Pseudocode
def store_object(content):
    hash = sha256(content)
    path = f".pes/objects/{hash[0:2]}/{hash[2:]}"
    write_file(path, content)
    return hash
```

This gives us:

- **Deduplication:** Identical files stored once
- **Integrity:** Hash verifies data isn't corrupted
- **Immutability:** Changing content = different hash = different object

Objects are sharded by the first two hex characters to avoid huge directories:

```
.pes/objects/
├── 2f/
│   └── 8a3b5c7d9e...
├── a1/
│   ├── 9c4e6f8a0b...
│   └── b2d4f6a8c0...
└── ff/
    └── 1234567890...
```

Exploring a Real Git Repository

You can inspect Git's internals yourself:

```
mkdir test-repo && cd test-repo && git init
echo "Hello" > hello.txt
git add hello.txt && git commit -m "First commit"

find .git/objects -type f          # See stored objects
git cat-file -t <hash>            # Show type: blob, tree, or commit
git cat-file -p <hash>            # Show contents
cat .git/HEAD                     # See what HEAD points to
cat .git/refs/heads/main          # See branch pointer
```

What You'll Build

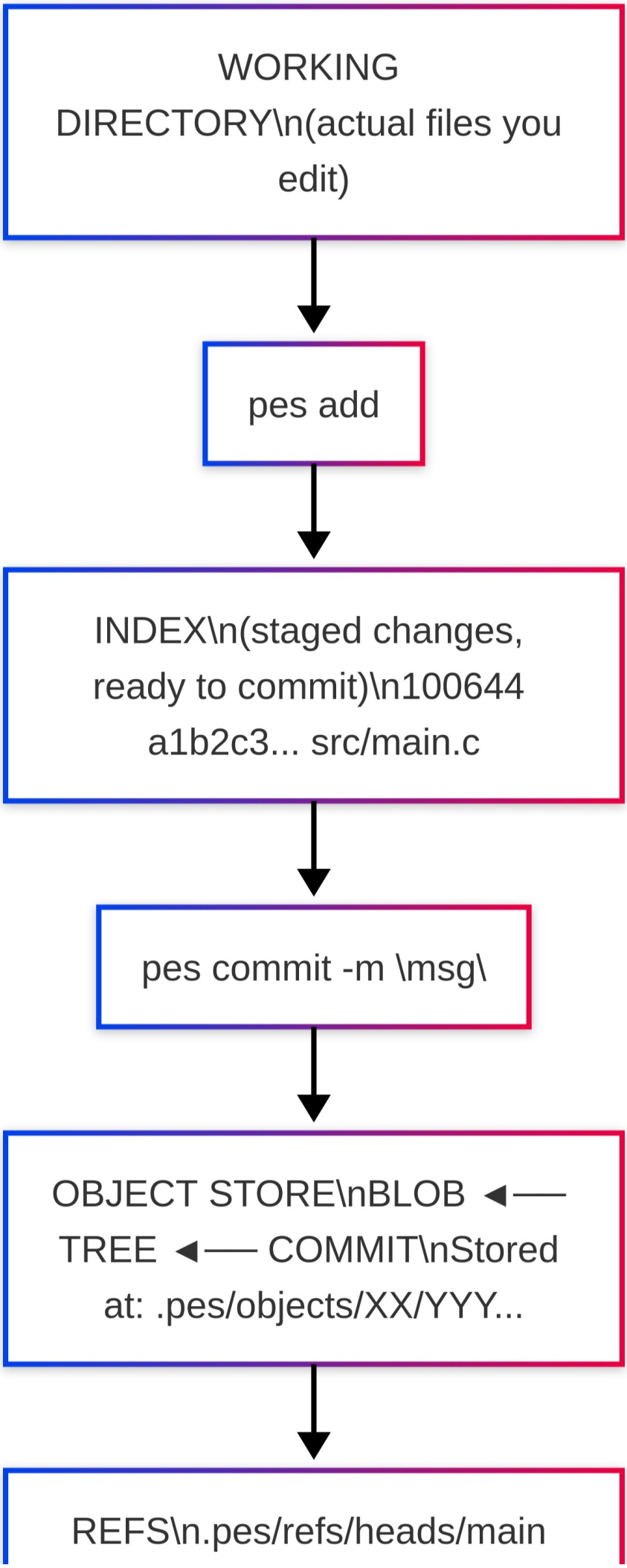
PES-VCS implements five commands across four phases:

```
pes init              Create .pes/ repository structure
pes add <file>...     Stage files (hash + update index)
pes status            Show modified/staged/untracked files
pes commit -m <msg>   Create commit from staged files
pes log               Walk and display commit history
```

The `.pes/` directory structure:

```
my_project/
├── .pes/
│   ├── objects/          # Content-addressable blob/tree/commit
storage
│   │   ├── 2f/
│   │   │   └── 8a3b...   # Sharded by first 2 hex chars of hash
│   │   └── a1/
│   │       └── 9c4e...
│   ├── refs/
│   │   └── heads/
│   │       └── main      # Branch pointer (file containing commit
hash)
│   ├── index             # Staging area (text file)
│   └── HEAD              # Current branch reference
└── (working directory files)
```

Architecture Overview



→ commit hash\n.pes/HEAD →
\ref: refs/heads/main\

Phase 1: Object Storage Foundation

Filesystem Concepts: Content-addressable storage, directory sharding, atomic writes, hashing for integrity

Files: `pes.h` (read), `object.c` (implement `object_write` and `object_read`)

What to Implement

Open `object.c`. Two functions are marked `// TODO`:

1. **`object_write`** - Stores data in the object store.
   - Prepends a type header (`"blob <size>\0"`, `"tree <size>\0"`, or `"commit <size>\0"`)
   - Computes SHA-256 of the full object (header + data)
   - Writes atomically using the temp-file-then-rename pattern
   - Shards into subdirectories by first 2 hex chars of hash
2. **`object_read`** - Retrieves and verifies data from the object store.
   - Reads the file, parses the header to extract type and size
   - **Verifies integrity** by recomputing the hash and comparing to the filename
   - Returns the data portion (after the `\0`)

Read the detailed step-by-step comments in `object.c` before starting.

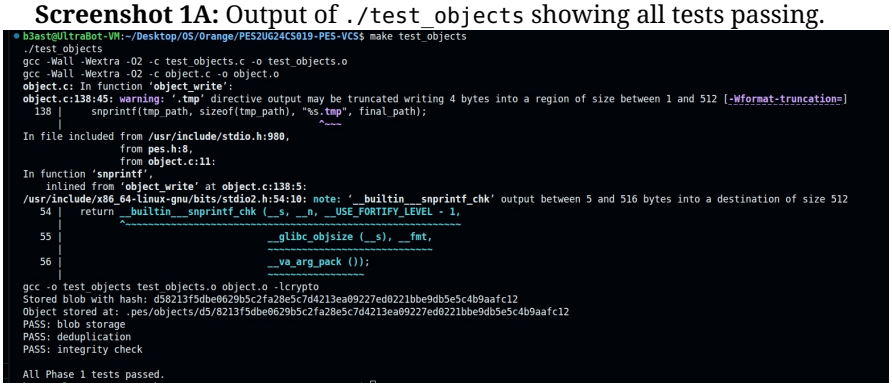
Testing

```
make test_objects
./test_objects
```

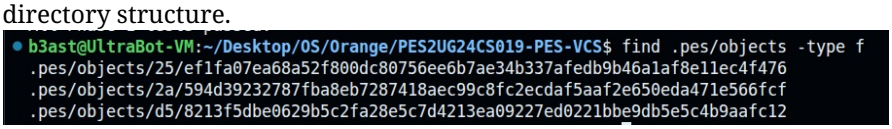
The test program verifies:

- Blob storage and retrieval (write, read back, compare)
- Deduplication (same content → same hash → stored once)
- Integrity checking (detects corrupted objects)

Screenshot 1A: Output of `./test_objects` showing all tests passing.



Screenshot 1B: `find .pes/objects -type f` showing the sharded directory structure.



Phase 2: Tree Objects

Filesystem Concepts: Directory representation, recursive structures, file modes and permissions

Files: `tree.h` (read), `tree.c` (implement all TODO functions)

What to Implement

Open `tree.c`. Implement the function marked `// TODO`:

1. **`tree_from_index`** - Builds a tree hierarchy from the index.
   - Handles nested paths: `"src/main.c"` must create a `src` subtree
   - This is what `pes commit` uses to create the snapshot
   - Writes all tree objects to the object store and returns the root hash

Testing

```
make test_tree
./test_tree
```

The test program verifies:

- Serialize → parse roundtrip preserves entries, modes, and hashes
- Deterministic serialization (same entries in any order → identical output)

Screenshot 2A: Output of `./test_tree` showing all tests passing.

```
b3ast@UltraBot-VM:~/Desktop/OS/Orange/PES2UG24CS019-PES-VCS$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from `find .pes/objects -type f` and run `xxd .pes/objects/XX/YYY... | head -20` to show the raw binary format.

```
===Screenshot 2B: raw tree object===

xxd: No such file or directory
Initialized empty PES repository in .pes/
Committed: 0bdcdaf07fa8... Initial commit
Committed: 08d83a7a0845... Add world
Committed: ba1525723d04... Add farewell
```

Phase 3: The Index (Staging Area)

Filesystem Concepts: File format design, atomic writes, change detection using metadata

Files: `index.h` (read), `index.c` (implement all TODO functions)

What to Implement

Open `index.c`. Three functions are marked `// TODO`:

1. **`index_load`** - Reads the text-based `.pes/index` file into an `Index` struct.
   - If the file doesn't exist, initializes an empty index (this is not an

error)
    - Parses each line: `<mode> <hash-hex> <mtime> <size> <path>`

2. **`index_save`** - Writes the index atomically (temp file + rename).
    - Sorts entries by path before writing
    - Uses `fsync()` on the temp file before renaming
3. **`index_add`** - Stages a file: reads it, writes blob to object store, updates index entry.
    - Use the provided `index_find` to check for an existing entry

`index_find`, `index_status` and `index_remove` are already implemented for you - read them to understand the index data structure before starting.

Expected Output of `pes status`

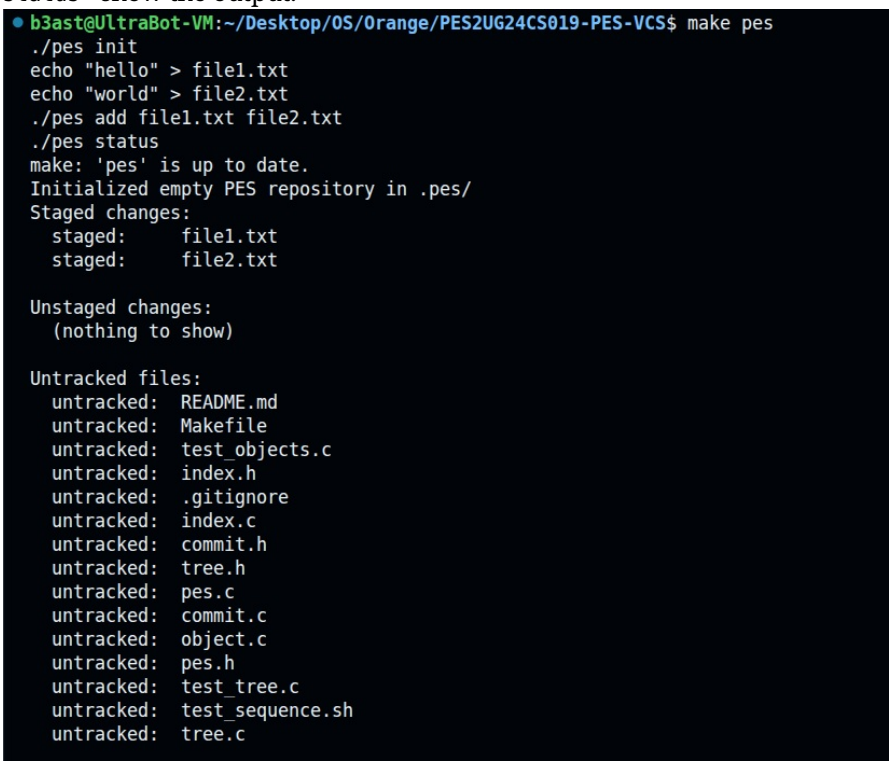
```
Staged changes:
  staged:     hello.txt
  staged:     src/main.c

Unstaged changes:
  modified:   README.md
  deleted:    old_file.txt

Untracked files:
  untracked:  notes.txt
```

If a section has no entries, print the header followed by `(nothing to show)`.

Testing

```
make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
```

Screenshot 3A: Run `./pes init`, `./pes add file1.txt file2.txt`, `./pes status` - show the output.



Screenshot 3B: `cat .pes/index` showing the text-format index with your entries.

```
b3ast@UltraBot-VM:~/Desktop/OS/Orange/PES2UG24CS019-PES-VCS$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776330810 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776330810 6 file2.txt
b3ast@UltraBot-VM:~/Desktop/OS/Orange/PES2UG24CS019-PES-VCS$
```

Phase 4: Commits and History

Filesystem Concepts: Linked structures on disk, reference files, atomic pointer updates

Files: `commit.h` (read), `commit.c` (implement all TODO functions)

What to Implement

Open `commit.c`. One function is marked `// TODO`:

1. **`commit_create`** - The main commit function:
   - Builds a tree from the index using `tree_from_index()` (**not** from the working directory - commits snapshot the staged state)
   - Reads current HEAD as the parent (may not exist for first commit)
   - Gets the author string from `pes_author()` (defined in `pes.h`)
   - Writes the commit object, then updates HEAD

`commit_parse`, `commit_serialize`, `commit_walk`, `head_read`, and `head_update` are already implemented - read them to understand the commit format before writing `commit_create`.

The commit text format is specified in the comment at the top of `commit.c`.

Testing

```
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
```

You can also run the full integration test:

```
make test-integration
```

Screenshot 4A: Output of `./pes log` showing three commits with hashes, authors, timestamps, and messages.

```
b3ast@UltraBot-VM:~/Desktop/OS/Orange/PES2UG24CS019-PES-VCS$  echo '===Screenshot 4A: pes log===' && ./pes log
===Screenshot 4A: pes log===
commit 9578cfee9487c8da9634f3d0e1c7faf85ec1d49c9407f12b430874fba92cdbdc
Author: PES User <pes@localhost>
Date:   1776331300

    Add file2

commit ff287560ae7691e76afb9ce38e4781ffc161cde684f5a3f25cb92b6b6a92a51a
Author: PES User <pes@localhost>
Date:   1776331300

    Update file1

commit 00c523a6ca2701aac0655ed6ce314f1824314fbab4c40b21d3a448effd84681b
Author: PES User <pes@localhost>
Date:   1776331300

    Initial commit
```

Screenshot 4B: `find .pes -type f | sort` showing object store growth after three commits.

```
===Screenshot 4B: object store growth===
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/08/d83a7a08452ec430acc1070e6f8a79f99bb1ddf06ec74377883ac20cba378b
.pes/objects/0b/dcdaf07fa8cc20ae3de0dec8497cdb161473d1449691fc3a0e20fdd5573296
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/ba/1525723d0465b0c754499a34bb9f7dbfe08460d7ec90a584b99c077356e783
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C: `cat .pes/refs/heads/main` and `cat .pes/HEAD` showing the reference chain.

```
===Screenshot 4C: refs chain===
ba1525723d0465b0c754499a34bb9f7dbfe08460d7ec90a584b99c077356e783
ref: refs/heads/main
b3ast@UltraBot-VM:~/Desktop/OS/Orange/PES2UG24CS019-PES-VCS$
```

Phase 5 & 6: Analysis-Only Questions

The following questions cover filesystem concepts beyond the implementation scope of this lab. Answer them in writing - no code required.

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` - what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

1. Resolve branch reference file: `.pes/refs/heads/<branch>`.
2. Read target commit hash from that file.
3. Read current commit hash from HEAD (through current ref if symbolic).
4. Parse both commit objects and load their root trees.
5. Build file maps for current-tree vs target-tree (path -> mode, hash).
6. Perform safety checks for uncommitted changes before touching files.
7. Update working directory to match target tree:
   - Delete tracked files present in current tree but absent in target tree.
   - Overwrite files whose blob hash differs.
   - Create new files/directories from target tree blobs.
   - Apply executable bit from mode.
8. Rewrite index to exactly match target tree snapshot.
9. Update `.pes/HEAD` to `ref: refs/heads/<branch>`.

Complexity comes from correctness and safety: conflict detection, recursive tree materialization, delete-vs-overwrite order, mode updates, and atomicity if checkout fails midway.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Use index as the expected baseline and object store as truth for staged content:

1. For each indexed path, `stat()` working file.

2. If file missing, mark dirty.
3. If mtime/size differs from index entry, re-hash file and compare with indexed blob hash.
4. If hash differs, file has unstaged changes.
5. Build diff between current-branch tree and target-branch tree.
6. If a dirty file is in the branch-diff set (modified/deleted/added incompatibly), refuse checkout.

So conflict condition is: dirty(path) AND branch_changes(path)

This prevents clobbering local edits.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD, new commits are created normally, but no branch ref is moved. HEAD points directly to a commit hash, so the new commits can become unreachable when HEAD moves elsewhere.

Recovery options:

1. While still detached, create a branch at current commit (`refs/heads/<name>` -> current hash).
2. If already moved away, recover via reflog-like history (or manual saved hash), then create a branch at that commit.

Creating a branch reattaches that line of history and prevents GC from collecting it.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects - blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Mark-and-sweep approach:

1. Roots = all branch heads in `.pes/refs/heads/*` (and tags if implemented).
2. Traverse each root commit graph (follow parent links).
3. For each visited commit, mark its tree hash reachable.
4. Recursively parse each tree; mark child trees and blob hashes reachable.
5. After marking, iterate all files under `.pes/objects/*/*`.
6. Delete any object hash not in reachable set.

Data structure: hash set of object IDs (O(1) average insert/lookup).

Scale estimate for 100,000 commits and 50 branches:

1. Commit visits: at most ~100,000 unique commits (branches usually overlap).
2. Tree/blob visits depend on churn; can be several times commits.
3. Total visited objects is typically in the low millions for medium projects, but upper bound is repository-size dependent.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Race example:

1. Commit process writes new tree/blob objects.
2. Before branch ref is updated, GC scans refs and does not see those new objects as reachable.
3. GC deletes them.
4. Commit process then writes commit object and updates ref to point to commit whose tree/blob was deleted.
5. Repository now references missing objects (corruption).

How Git avoids this:

1. Uses lock files and reference transaction semantics around ref updates.
2. Uses temporary object retention and grace periods (reflog-based reachability windows).
3. Runs GC with coordination so newly created objects are not reclaimed prematurely.

Submission Checklist

Screenshots Required

Phase	ID	What to Capture
1	1A	`./test_objects` output showing all tests passing
1	1B	`find .pes/objects -type f` showing sharded directory structure
2	2A	`./test_tree` output showing all tests passing
2	2B	`xxd` of a raw tree object (first 20 lines)
3	3A	`pes init` → `pes add` → `pes status` sequence
3	3B	`cat .pes/index` showing the text-format index
4	4A	`pes log` output with three commits
4	4B	`find .pes -type f | sort` showing object growth
4	4C	`cat .pes/refs/heads/main` and `cat .pes/HEAD`
Final	5	Screenshot 5: Full integration test (`make test-integration`)

Code Files Required (5 files)

File	Description
`object.c`	Object store implementation
`tree.c`	Tree serialization and construction
`index.c`	Staging area implementation
`commit.c`	Commit creation and history walking

Analysis Questions (written answers)

Section	Questions
Branching (analysis-only)	Q5.1, Q5.2, Q5.3
GC (analysis-only)	Q6.1, Q6.2

Submission Requirements

1. GitHub Repository

- You must submit the link to your GitHub repository via the official submission link (which will be shared by your respective faculty).

- The repository must strictly maintain the directory structure you built throughout this lab.
- Ensure your github repository is made `public`

2. Lab Report

- Your report, containing all required **screenshots** and answers to the **analysis questions**, must be placed at the **root** of your repository directory.
- The report must be submitted as either a PDF (`report.pdf`) or a Markdown file (`README.md`).

3. Commit History (Graded Requirement)

- **Minimum Requirement:** You must have a minimum of **5 commits per phase** with appropriate commit messages. Submitting fewer than 5 commits for any given phase will result in a deduction of marks.
- **Best Practices:** We highly prefer more than 5 detailed commits per phase. Granular commits that clearly show the delta in code block changes allow us to verify your step-by-step understanding of the concepts and prevent penalties <3

Further Reading

- **Git Internals** (Pro Git book): https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain
- **Git from the inside out**: https://codewords.recurse.com/issues/two/git-from-the-inside-out
- **The Git Parable**: https://tom.preston-werner.com/2009/05/19/the-git-parable.html